

IMT Atlantique

Technopôle de Brest-Iroise - CS 83818

29238 Brest Cedex 3

URL : www.imt-atlantique.fr



Plan du livrable Rapport du projet

PRONTO-2025-18

Conception d'un Agent IA pour l'Observatoire Astronomique

Membres de groupe :

Mohamed-Ali BOULLOUZ,

Axel JONARD,

Brahim ONASSER,

Ayman OUNZAR

Encadrants :

Thomas BOUTEYRON,

Dominique FABRE,

Bastien PASDELOUP

Date d'édition : 19 mai 2025

Version : 1.0



IMT Atlantique

Bretagne-Pays de la Loire

École Mines-Télécom

Résumé

Ce projet vise à développer un agent conversationnel intelligent, basé sur Mistral.AI, afin de guider les utilisateurs d'un observatoire astronomique dans l'utilisation du télescope. L'agent, combinant une approche RAG (Retrieval Augmented Generation) avec une interface Streamlit, interagit exclusivement à partir de documents fournis. L'objectif est de fournir un outil fiable, rapide et ancré dans la documentation technique. Ce rapport retrace les étapes du projet, de la conception à l'intégration, en passant par la planification, les choix technologiques, et les solutions mises en œuvre face aux difficultés.

Sommaire

1. Introduction	2
1.1. Contexte et enjeux	2
1.2. Problématique	2
1.3. Objectifs	2
1.4. Annonce de plan	2
2. Conception	2
2.1. Rappel des fonctionnalités	2
2.2. Choix de conception	3
3. Réalisation et Développement	4
3.1. Architecture	5
3.2. Traitement des documents	5
3.3. Création et développement l'interface utilisateur	7
3.4. Tests	7
3.4.1. Objectif et jeu de test	7
3.4.2. Génération des paires (question, réponse)	7
3.4.3. Pipeline expérimental	8
3.4.4. Présentation des résultats	9
3.4.5. Interprétation des résultats	12
4. Intégration et Validation	13
4.1. Intégration des composants	13
4.2. Scénarios de validation	14
4.3. Pistes d'amélioration	19
5. Conclusions et perspectives	19
5.1. Organisation et planification	19
5.2. Bilan	19
5.3. Perspectives	19
Bibliographie	19
Annexes	20
Annexe 1 – Retour d'expérience individuel	20

1. Introduction

1.1. Contexte et enjeux

L'association "Gens de la Lune" souhaite renforcer l'accessibilité de son observatoire astronomique au grand public. En effet, l'utilisation du matériel d'observation, notamment le télescope, reste souvent complexe pour les personnes non initiées. Pour faciliter l'expérience des visiteurs, l'association envisage la mise en place d'un assistant intelligent capable d'accompagner les usagers dans la manipulation des équipements, de répondre à leurs questions.

1.2. Problématique

Comment concevoir un agent intelligent interactif, capable d'assister efficacement les utilisateurs dans l'utilisation d'un télescope astronomique, en s'appuyant uniquement sur la documentation existante, afin de rendre l'observation accessible, autonome et enrichissante pour un public non expert ?

1.3. Objectifs

- Fournir une assistance technique : Répondre de manière claire et précise aux questions des utilisateurs concernant le fonctionnement et les réglages du télescope, en s'appuyant sur la documentation existante.
- Accompagner l'utilisateur pas à pas : Proposer un guidage interactif et progressif pour la prise en main du télescope, depuis l'installation jusqu'à l'observation.
- Détecter et réagir aux situations d'erreur ou de panne : Identifier les problèmes courants rencontrés par les utilisateurs et suggérer des solutions adaptées ou des diagnostics.
- Offrir une interface intuitive et accessible : Concevoir une interface simple, ergonomique et adaptée à un public non expert, facilitant l'interaction avec l'agent IA.

1.4. Annonce de plan

Le rapport débute par la conception du système, avec un rappel des fonctionnalités attendues de l'agent intelligent et un exposé des choix techniques et architecturaux retenus. Il enchaîne avec la phase de réalisation et de développement, qui comprend le traitement des documents, la création d'une interface utilisateur intuitive, ainsi que la mise en place d'un protocole de tests. La suite du rapport aborde l'intégration des différents composants de la solution et leur validation à travers des scénarios d'usage réels. Enfin, un bilan du projet est dressé, suivi de pistes d'amélioration et de perspectives d'évolution pour enrichir ou généraliser le système.

2. Conception

2.1. Rappel des fonctionnalités

- FP1 : Assistance conversationnelle contextualisée
- FP2 : Assistance à la résolution de pannes
- FS1 : Rapidité (reste encore relatif au modèle IA non payant que nous utilisons.)
- FS2 : Permettre aux utilisateurs d'accéder à l'historique de leurs conversations avec le chatbot, propre à leur compte, et offrir aux responsables de l'observatoire un accès à l'ensemble des échanges les autres historiques de conversation des autres utilisateurs.
- FC1 : Accès permanent aux documents référencés dans la base de données du chatbot, assurant des réponses fondées sur des sources fiables
- FC2 : Une interface intuitive, claire et bien structurée, permettant une interaction fluide avec le chatbot
- FC3 : Le chatbot est facilement adaptable et modifiable afin d'intégrer les évolutions du matériel de l'observatoire ainsi que les éventuels changements dans la documentation, grâce à des boutons de saisie intégrés à l'interface.

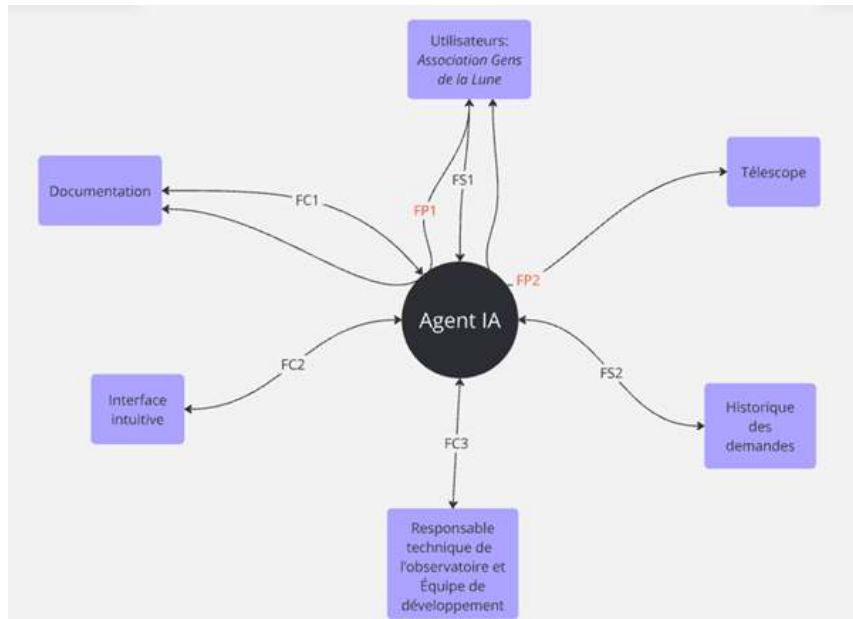


FIGURE 1 – Illustration des fonctionnalités principales de l’agent IA

2.2. Choix de conception

Comme déjà signalé, l’agent conversationnel est conçu pour répondre aux questions des utilisateurs concernant l’observatoire astronomique de l’école IMT Atlantique, campus de Brest, en s’appuyant uniquement sur une base documentaire locale et en formulant ses réponses en français.

Il s’agit d’un agent RAG (Retrieval-Augmented Generation), combinant des capacités de recherche d’information dans une base documentaire avec la génération de texte, afin de fournir des réponses précises, contextualisées et fondées sur les documents disponibles.

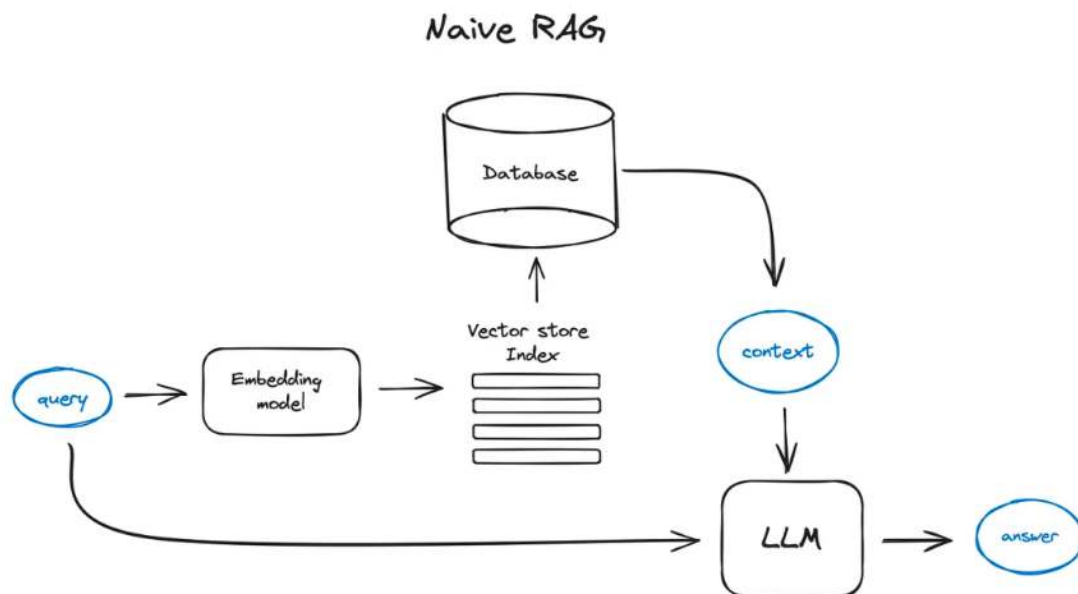


FIGURE 2 – Comment fonctionne un agent RAG

Nous avons choisi d’utiliser les outils de Mistral AI, comme suggéré dans le sujet du projet. Voici les principaux choix de conception de cet agent conversationnel, accompagnés de leurs justifications

techniques :

- **Utilisation de LangChain** pour orchestrer les chaînes de traitement des requêtes (retrieval, prompt engineering, génération).
Ce framework facilite la modularisation de l'architecture du chatbot. Il offre une abstraction efficace des étapes classiques d'un système RAG (Retrieval-Augmented Generation) et permet une intégration fluide entre sources de données, modèles d'embedding, moteurs vectoriels et LLM. Cela a permis de structurer le pipeline en composants réutilisables et testables.
- **Intégration de Mistral AI** comme LLM via les modules ChatMistralAI et MistralAIEmbeddings.
Le choix de Mistral AI se justifie d'abord par son inclusion explicite dans le cahier des charges du projet. En second lieu, Mistral propose une API accessible via un plan expérimental gratuit, ce qui permet de tester ses modèles de langage et d'embedding sans contrainte financière, tout en garantissant une qualité de génération adaptée à notre cas d'usage.
- **Choix du modèle mistral-large-latest** Le modèle mistral-large-latest représente actuellement le modèle le plus performant proposé par Mistral AI. Son choix s'impose naturellement pour garantir une qualité optimale dans la compréhension des requêtes et la génération de réponses cohérentes et informatives. Pour les embeddings, nous avons utilisé mistral-embed, qui est à ce jour l'unique modèle d'encodage sémantique mis à disposition par Mistral. L'utilisation de ces deux modèles s'est faite exclusivement via l'API officielle, car leur exécution en local nécessiterait une infrastructure matérielle très puissante, notamment pour mistral-large-latest qui comporte 124 milliards de paramètres.
- **Utilisation de FAISS** pour l'indexation des documents de l'observatoire.
FAISS (Facebook AI Similarity Search) est une bibliothèque open source développée pour la recherche rapide de similarité sur des vecteurs de grande dimension. Elle permet d'effectuer des requêtes de type *nearest neighbors* à haute performance, même sur de grands volumes de textes vectorisés. Son intégration avec LangChain et sa capacité à être sauvegardée localement en font une solution idéale pour un système embarqué ou déployé sur serveur local.

3. Réalisation et Développement

3.1. Architecture

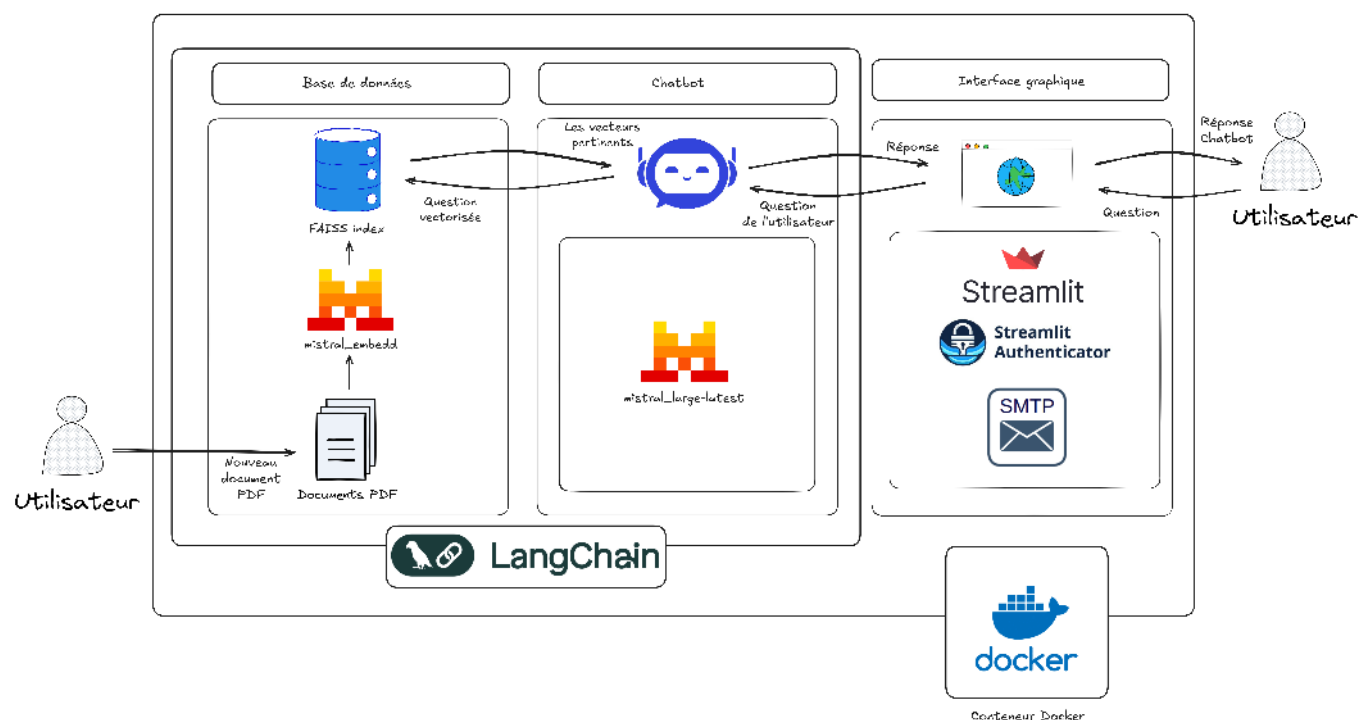


FIGURE 3 – Architecture du chatbot

3.2. Traitement des documents

L'embedding des documents dans un agent RAG (Retrieval-Augmented Generation) avec LangChain consiste à transformer des textes en vecteurs numériques à l'aide d'un modèle d'embedding. Ces vecteurs représentent le sens des textes et sont stockés dans une base de vecteurs (`faiss index` -voir le code-). Lorsqu'un utilisateur pose une question, cette dernière est également convertie en vecteur, puis comparée aux documents déjà embeddés pour retrouver ceux qui sont sémantiquement les plus proches. Les documents ainsi sélectionnés sont utilisés comme contexte pour générer une réponse pertinente avec un modèle de langage.

Lorsque les documents contiennent des pages scannées, des images ou des tableaux complexes, une simple extraction du texte brut ne suffit plus. C'est pourquoi trois classes progressives ont été conçues pour s'adapter à ces cas de figure croissants en complexité : `TextEmbedder`, `Embedder`, `EmbedderWithOcr`, et `MultimodalEmbedder` :

1. `Embedder` – squelette générique et gestionnaire d'index :

Cette classe est le **cœur de l'architecture**. Elle permet de gérer tout le cycle de vie de la vectorisation (création, mise à jour, suppression et interrogation d'un index FAISS) indépendamment de la façon dont le texte est obtenu.

2. `TextEmbedder` – extraction texte « PDF natif » :

Cette classe extrait uniquement le texte brut lisible des PDF et le segmente intelligemment en morceaux (chunks) via `RecursiveCharacterTextSplitter` de LangChain, préparant ainsi le contenu à une vectorisation efficace. Elle utilise notamment :

- `PyPDFLoader` pour charger les fichiers PDF,
- FAISS comme vector stores.

Cette approche est idéale pour les documents structurés, sans éléments graphiques complexes.

3. `EmbedderWithOcr` – traitement enrichi par OCR :

Héritant de `Embedder`, la classe `EmbedderWithOcr` intègre une couche de reconnaissance optique

de caractères (OCR), ce qui lui permet d'extraire du texte à partir des images contenues dans les PDF. Elle combine :

- PyMuPDF pour l'extraction des images de pages PDF,
- pdf2image pour la conversion des pages en images exploitables,
- Tesseract OCR (via pytesseract) pour la reconnaissance du texte. Cette classe fusionne intelligemment le texte OCR avec le texte natif pour fournir un contenu complet, particulièrement utile dans les cas de documents scannés ou hybrides.

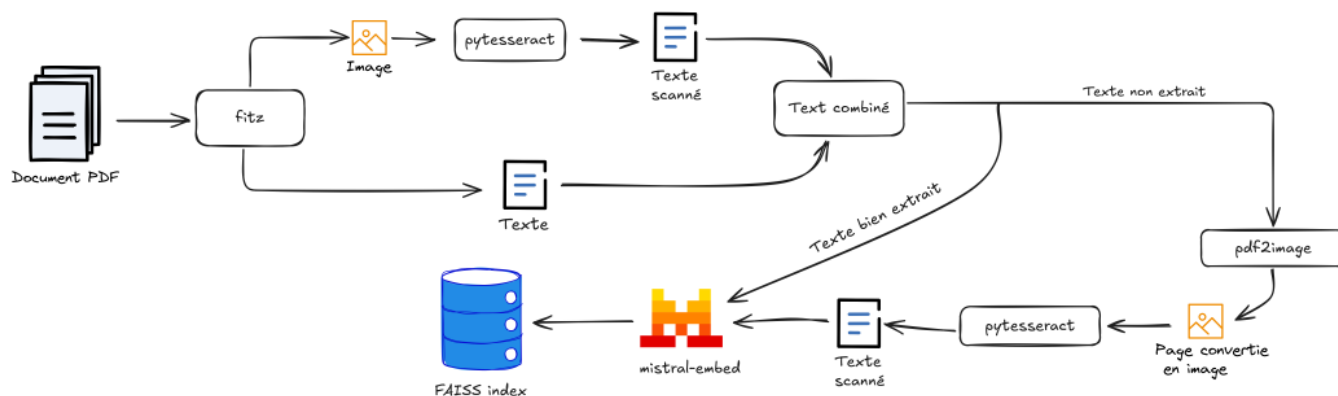


FIGURE 4 – Fonctionnement de l'EmbedderWithOcr

4. MultimodalEmbedder – intelligence sémantique multimodale :

La classe MultimodalEmbedder représente le niveau le plus avancé de traitement, en intégrant une approche multimodale pour exploiter à la fois le texte, les images, et les tableaux présents dans un document. Elle repose sur la bibliothèque unstructured pour segmenter finement les éléments du document (titres, paragraphes, figures, tableaux, etc.).

Elle se distingue par sa capacité à transformer les contenus visuels en représentations textuelles exploitables sémantiquement, en s'appuyant sur des modèles de langage avancés à savoir et qu'on a utilisé dans ce projet.

Deux modèles principaux sont utilisés ici :

- Le modèle mistral-large-latest (via ChatMistralAI) pour résumer les tableaux en texte clair et concis (summarize_Table).
- et surtout, pixtral-large-latest, un modèle multimodal (image-texte) développé par Mistral, utilisé dans summarize_image. Ce modèle est capable de décrire les images en langage naturel à partir d'un prompt construit à partir du contexte textuel voisin de l'image.

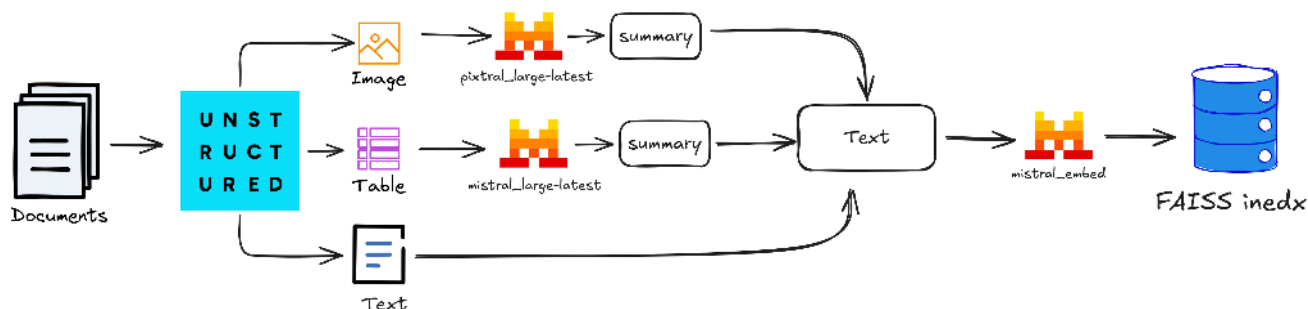


FIGURE 5 – Fonctionnement de MultimodalEmbedder

3.3. Création et développement l'interface utilisateur

L'interface utilisateur de notre agent IA a été développée à l'aide de la bibliothèque `streamlit`, un framework Python open-source qui permet de créer rapidement des applications web interactives. `streamlit` se distingue par sa simplicité d'utilisation et son intégration fluide avec les scripts Python classiques.

Dans un souci d'ergonomie et d'accessibilité, nous avons fait le choix de concevoir dès le départ une interface intuitive et minimaliste, afin de garantir une expérience utilisateur claire et fluide. Cette approche nous a permis de nous concentrer sur l'essentiel : une interaction simple et efficace entre l'utilisateur et l'agent d'intelligence artificielle.

Grâce aux nombreux tutoriels disponibles sur YouTube, ainsi qu'à la documentation fournie par `streamlit`, nous avons pu progresser étape par étape dans le développement de l'interface. Cela nous a permis d'ajouter progressivement de nouvelles fonctionnalités, rendant l'interface de plus en plus polyvalente et adaptée aux besoins du projet.

La communication entre l'interface et l'agent IA repose alors sur des fonctions Python dédiées. Lorsqu'une requête est soumise, elle est transmise à l'agent IA via une fonction passerelle, qui gère l'appel du modèle et retourne la réponse générée. Cette réponse est ensuite affichée dans l'interface de manière claire et instantanée. Ce mécanisme assure une interaction fluide et réactive, tout en maintenant une séparation claire entre la logique de traitement et la couche d'affichage.

3.4. Tests

3.4.1. Objectif et jeu de test

L'objectif de cette campagne d'évaluation est de mesurer, de manière reproductible, la qualité des réponses produites par un chatbot basé sur un pipeline *Retrieval-Augmented Generation (RAG)*¹.

Le jeu de test se compose de 80 paires (**question, réponse de référence, nom de document**), réparties équitablement sur 8 documents PDF techniques (10 paires par document).

La méthodologie est inspirée des approches proposées par [Hugging Face](#) et [LangChain](#).

3.4.2. Génération des paires (question, réponse)

Les paires de test ont été générées par un autre LLM que celui utilisé pour le chatbot évalué. Le modèle retenu est **gpt-o3**, reconnu pour ses performances élevées en compréhension et traitement de documents.

Méthodologie

Chaque document est chargé dans une nouvelle session ChatGPT. Le prompt suivant est utilisé pour générer les paires :

1. Génération : `mistral-large-latest` ; recherche : `FAISS + mistral-embed`.

SYSTEM :

You are a data-curation assistant. Your task is to create a **high-quality evaluation set for a Retrieval-Augmented-Generation (RAG) system** that answers questions about the IMT Atlantique campus-Brest astronomical observatory.

DOMAIN CONTEXT : The document contains information about the equipment of the astronomical observatory and how to use them. Questions must target such *specific* knowledge—never general astronomy trivia.

INPUT : You will receive **one attached PDF**. Use its file name verbatim for the field `doc_name`. Assume you have the full OCR text.

OUTPUT : Return **only** a JSON array with exactly ten objects :

```
{  
  "question": "<French question>",  
  "answer": "<French answer, strictly supported by the document>",  
  "doc_name": "<attached-file.pdf>"  
}
```

RULES :

- **Not detailed questions** – The questions shouldn't be too detailed ; imitate the questions that users of the observatory would ask.
- **Detailed answers** – The answers you provide should have as many details as possible, details that only a chatbot that has access to the document can provide.
- **Unique knowledge** – Ask about facts, numbers, procedures or hardware details that a generic LLM would not know without this document.
- **Answer support** – Every answer must be directly verifiable in the PDF (quote-level support); do not invent information.
- **Difficulty mix** – Include both straightforward look-ups and questions that require combining two passages.
- **Coverage** – The ten questions must cover distinct parts of the document (no duplicates).
- **Language** – Write both questions and answers in French.
- **LaTeX** – Enclose any formula in dollar signs.
- **Sources** – Do not add citations or markdown ; output strictly the JSON array.

BEGIN.

Dans une seconde étape, chaque ensemble généré est validé à l'aide d'une nouvelle requête à `gpt-o3` pour vérifier la conformité au cahier des charges. Les paires non conformes sont automatiquement corrigées.

Cette procédure garantit que le jeu de test est de haute qualité, adapté à l'évaluation rigoureuse d'un système RAG.

3.4.3. Pipeline expérimental

L'évaluation repose sur la méthode **LLM-as-a-Judge**, qui consiste à faire évaluer les réponses du chatbot par un autre LLM.

Mesures évaluées

- **Correcteness (justesse factuelle)** : évalue la précision de la réponse par rapport à la réponse de référence. Une note élevée signifie qu'aucune information erronée (chiffre, formule, procédure) n'a été introduite.
- **Faithfulness (fidélité aux documents)** : indique dans quelle mesure les informations fournies proviennent réellement des documents récupérés. Un score élevé montre l'absence d'hallucinations ; chaque fait est appuyé par le contexte.
- **Relevance (pertinence)** : mesure le degré d'adéquation entre la réponse et la question posée. Une valeur forte indique que le chatbot répond exactement au sujet, sans digressions ni contenu hors-sujet.

- **Semantic similarity (similarité sémantique)** : calcule (par cosinus) la proximité conceptuelle entre la réponse du chatbot et la réponse de référence. Plus le score se rapproche de 1, plus les deux textes véhiculent les mêmes idées clés.
- **Latency (latence)** : temps écoulé (en secondes) pour produire la réponse complète. Une latence faible correspond à une interaction rapide et donc à une meilleure expérience utilisateur.

Protocole de notation

Pour **Correctness**, **Faithfulness** et **Relevance**, le modèle `mistral-large-latest` a été utilisé pour noter chaque réponse sur une échelle de 1 à 5. Un prompt spécifique a été conçu pour chaque critère afin de guider l'évaluation de manière fiable.

La **similarité sémantique** a été calculée avec le modèle `all-MiniLM-L6-v2`. Cette mesure est entièrement objective, car elle repose uniquement sur des embeddings et ne dépend pas du jugement d'un LLM.

La **latence** est mesurée automatiquement lors de la génération de la réponse par le chatbot, en calculant la durée entre l'envoi de la requête et la réception de la réponse.

Enfin, la fonctionnalité d'**historique de conversation** a été désactivée pour garantir que chaque réponse est produite indépendamment, sans contexte de tour précédent.

3.4.4. Présentation des résultats

TABLE 1 – Moyennes par document et par métrique (RAG / no-RAG)

Document	Latency(s)		Correct		Faithfulness		Relevance		semantic similarity	
	Sans RAG	Avec RAG	Sans RAG	Avec RAG	Sans RAG	Avec RAG	Sans RAG	Avec RAG	Sans RAG	Avec RAG
Le-coronographe-de-Lyot.pdf	4.485	9.432	3.100	4.100	3.700	4.800	4.600	4.900	0.647	0.703
Le-telescope-de-Schmidt-Cassegrain.pdf	3.158	20.414	2.200	4.800	2.300	5.000	3.700	4.700	0.535	0.708
Manuel de formation - Telescope.pdf	1.983	29.173	1.800	4.600	2.400	4.900	3.100	4.900	0.572	0.764
Pointage-et-suivi-automatiques.pdf	2.043	32.705	1.600	4.100	2.800	4.900	2.700	4.300	0.473	0.705
Syno_UsersGuide_NAServer_fra.pdf	1.735	8.437	1.700	5.000	1.700	4.900	1.900	4.300	0.461	0.807
astro-procedures-resume-anon.pdf	2.576	8.428	1.800	2.900	1.800	4.300	3.700	3.500	0.533	0.608
méthode-de-Bigourdan.pdf	3.240	17.499	1.800	3.700	2.500	4.100	3.700	4.200	0.498	0.540
stf-8300_manual_12.pdf	0.792	36.316	0.800	3.400	1.800	4.000	1.300	3.600	0.313	0.622

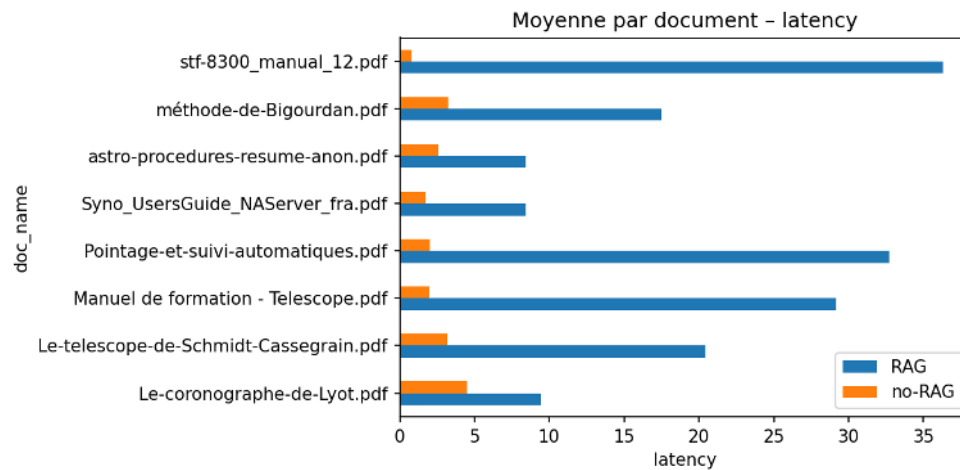


FIGURE 6 – Comparaison de la latence entre les deux approches : avec RAG et sans RAG

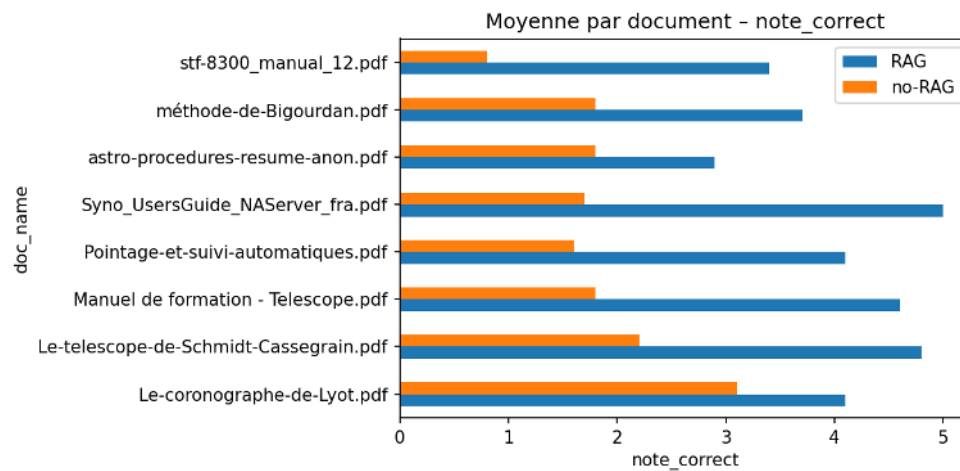


FIGURE 7 – Comparaison de la justesse entre les deux approches : avec RAG et sans RAG

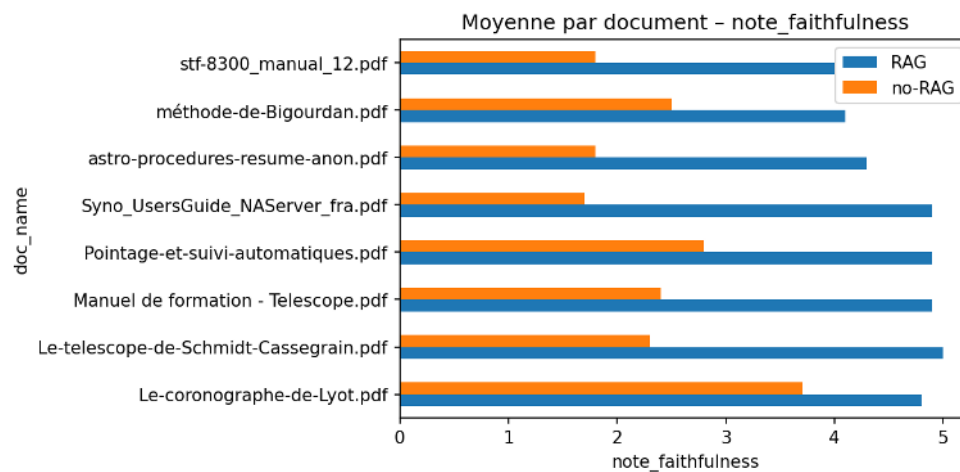


FIGURE 8 – Comparaison de la fidélité entre les deux approches : avec RAG et sans RAG

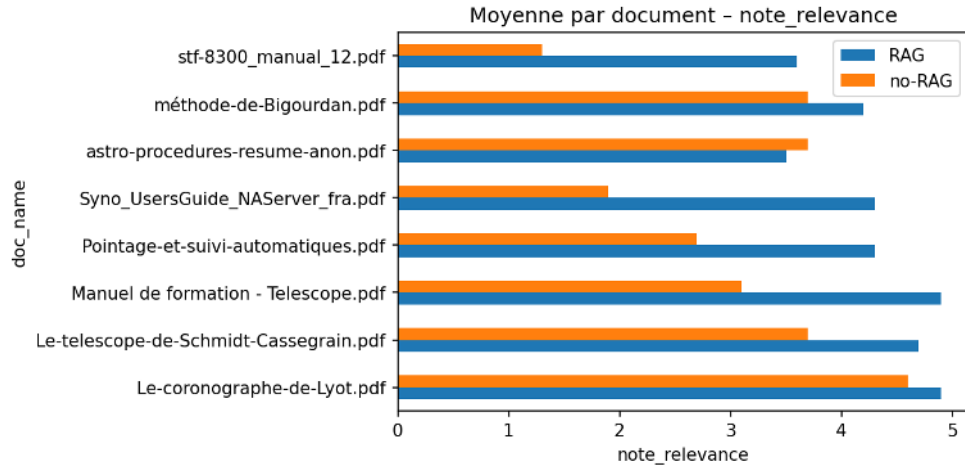


FIGURE 9 – Comparaison de la pertinence entre les deux approches : avec RAG et sans RAG

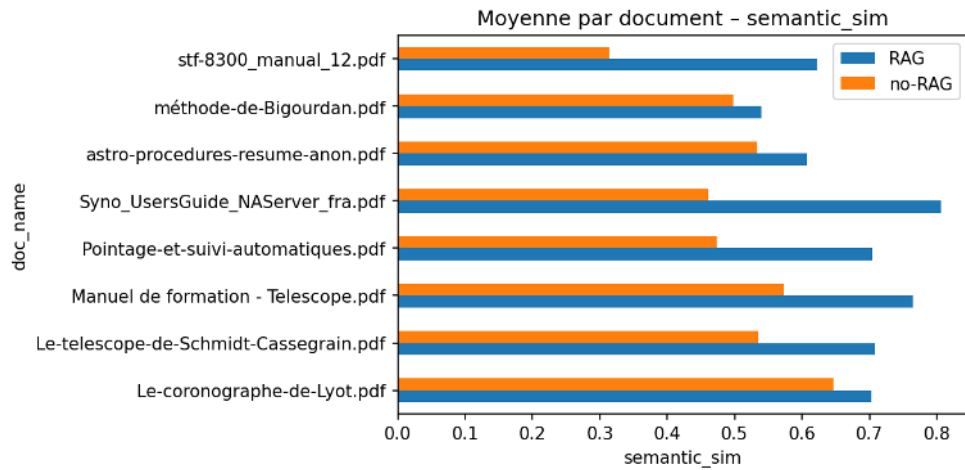


FIGURE 10 – Comparaison de la similarité sémantique entre les deux approches : avec RAG et sans RAG

3.4.5. Interprétation des résultats

▪ Moyennes globales :

	Latency(s) ↓	Correctness ↑	Relevance ↑	Faithfulness ↑	Semantic similarity ↑
Sans RAG	2.50	1.85	3.09	2.38	0.504
Avec RAG	20.30	4.07	4.30	4.61	0.682
$\Delta(\text{RAG} - \text{Sans RAG})$	+17.80	+2.22	+1.21	+2.24	+0.178
$\Delta (\%)$	+712 %	+120 %	+39 %	+94 %	+35 %

TABLE 2 – Moyennes globales pour chaque métrique

Les quatre métriques de qualité s'améliorent nettement ($\approx +35\%$ à $+120\%$) avec le RAG. Toutefois, le temps de réponse est multiplié par 8, ce qui constitue un compromis important.

▪ Analyse par document :

1. **Qualité :** Sur les huit documents, le chatbot avec le RAG dépasse celui sans RAG :
 - 8/8 fois pour *Correctness*, *Faithfulness*, et *Semantic similarity*;

— 7/8 fois pour *Relevance* (exception : *astro-procedures-resume.pdf*, gain marginal du chatbot sans RAG).

Les gains sont particulièrement visibles sur les documents techniques où le modèle sans RAG produit des hallucinations. L'ajout de passages contextuels permet au modèle de rester ancré dans la vérité du document (e.g. gain de +3.3 points en *Correctness* pour *Synology NAS*).

2. **Latence** : RAG est plus lent pour tous les documents (moyenne de +17.8 s), principalement à cause du coût de la recherche vectorielle et du reranking.

■ Interprétation :

1. **Efficacité du RAG**. Les scores dépassant 4/5 en *Correctness* et *faithfulness* montrent que le chatbot restitue fidèlement les contenus. La similarité sémantique augmente de 0.50 à 0.68, traduisant une meilleure cohérence tout en conservant des reformulations utiles.
2. **Coût en latence**. Bien que le temps de réponse soit allongé, il reste acceptable. Une optimisation du pipeline est cependant nécessaire pour un usage en production.

■ Conclusion :

Le couplage **LLM + RAG** améliore significativement la qualité des réponses sur des documents techniques, au prix d'un surcoût de latence.

4. Intégration et Validation

4.1. Intégration des composants

Le code source du chatbot est présent dans la page GitHub : https://github.com/OUNZAR-Ayman/PRONTO18-ai_agent.

1. Code applicatif : `app/`

Le cœur du projet se trouve dans le dossier `app/`. On y retrouve l'interface Streamlit (`interface.py`), les chaînes LangChain qui implémentent la logique RAG (`chat_bot.py`) ainsi que toutes les classes d'« embedder ». Sans ce répertoire, le chatbot n'existe tout simplement pas : il regroupe la totalité des algorithmes, des appels Mistral et de la gestion d'état.

2. Configuration sensible : `config.yaml` et `.env`

`config.yaml` contient les identifiants hachés pour `streamlit-authenticator` et la clé de signature du cookie ; `.env` stocke les clés API (Mistral, Hugging Face, SMTP). Ces fichiers commandent l'authentification et l'accès aux services externes ; sans eux, l'application refuse de démarrer. Le gabarit public `config.example.yaml` sert uniquement de modèle aux utilisateurs.

3. Stockage vectoriel et documents : `faiss_index/` et `docs/`

Le répertoire `faiss_index/` héberge l'index FAISS persistant, tandis que `docs/` conserve les PDF chargés définitivement. Ensemble, ils représentent la base de connaissances du chatbot ; supprimer ces dossiers revient à effacer toute la mémoire documentaire.

4. Déploiement conteneurisé : `Dockerfile` et `docker-compose.yml`

Le `Dockerfile` construit une image autonome (Python + Streamlit + dépendances), et `docker-compose.yml` orchestre le service, en montant des volumes pour `docs/` et `faiss_index/`. Ils garantissent la reproductibilité en production et l'isolation de l'environnement.

5. Index auxiliaire : `document_index.json`

Ce fichier fait le pont entre le nom d'un PDF et son identifiant interne (`doc_id`) utilisé dans FAISS. Il permet de supprimer proprement tous les vecteurs associés à un document donné, sans ré-embarquements inutiles.

6. Historique utilisateur : `chat_histories/`

Chaque conversation est sauvegardée au format JSON dans un sous-dossier propre à l'utilisateur. Cela offre une fonction de reprise de session et un audit complet des échanges.

7. Tests : `tests/`

Ce dossier contient les scripts Python permettant d'évaluer les performances du chatbot. Ces scripts ont été utilisés pour produire les résultats présentés dans la section *Tests* de ce rapport. On y trouve également un sous-dossier `plots/`, qui regroupe les figures générées automatiquement, notamment les histogrammes représentant les scores des différents aspects évalués. Cette structure facilite l'analyse reproductible et visuelle des performances du système.

8. Fichiers d'exclusion : `.gitignore` et `.dockerignore`

Ces listes protègent le dépôt Git : pas de secrets versionnés, pas d'artefacts volumineux ou inutiles intégrés dans l'image Docker.

9. Ressources statiques : `assets/`

Icônes, captures et illustrations destinées à l'interface ou à la documentation ; indispensables pour l'aspect visuel, mais non critiques pour la logique métier.

10. Documentation et dépendances : `README.md` et `requirements.txt`

Le `README.md` explique l'installation, la configuration et l'utilisation, tandis que `requirements.txt` dresse la liste exhaustive des bibliothèques Python nécessaires (Streamlit, LangChain, FAISS, Mistral SDK, Tenacity, etc.). Ils complètent l'écosystème sans influencer directement sur l'exécution du chatbot.

4.2. Scénarios de validation

Scénario 1 – Gestion d'un problème matériel

Objectif : Simuler une situation d'erreur pour tester la robustesse du chatbot face à un problème technique matériel.

Question posée :

Le télescope ne se déplace pas même si j'ai lancé Prism, que faire ?

Réponse attendue :

Le bot suggère de vérifier la connexion au lien matériel dans Prism, l'alimentation via la multiprise (bouton vert), puis la position de parking du télescope.

Commentaire :

Ce test valide la capacité du chatbot à orienter un diagnostic et proposer des étapes logiques de résolution.

Réponse du chatbot :



Si le télescope ne se déplace pas même après avoir lancé Prism, voici les étapes à suivre pour résoudre le problème :

1. Vérifier l'alimentation :

- Assurez-vous que l'onduleur, l'alimentation et le MCMT sont sous tension.
- Vérifiez que le bouton vert de la multiprise dans l'armoire près de l'ordinateur est allumé.

2. Établir le lien matériel :

- Dans l'onglet « observatoire » de la fenêtre principale de Prism, cliquez sur « établir lien matériel télescope ».
- Une fenêtre s'affichera pour choisir la position initiale du télescope. Sélectionnez la bonne position (tube retourné ou à l'Est, ou tube non retourné ou à l'Ouest) et cliquez sur « OK ».

3. Initialisation et synchronisation :

- Dans Prism, allez dans le menu « Contrôle télescope » et sélectionnez « Init. Reset ».
- Assurez-vous que la position du télescope n'est pas en "parking". Si c'est le cas, changez la position du télescope à "parking".
- Utilisez la carte du ciel pour pointer sur l'objet que vous souhaitez observer, faites un double-clic puis synchronisez avec l'objet.

4. Vérifier le suivi :

- Dans le menu « Contrôle coupole », assurez-vous que l'option « activer suivi coupole » est cochée.

5. Vérifier les connexions physiques :

- Assurez-vous que tous les câbles sont correctement branchés et que le « coup de poing » d'arrêt d'urgence n'est pas enclenché.

6. Redémarrer Prism :

- Si le problème persiste, essayez de redémarrer Prism et répétez les étapes ci-dessus.

Si après avoir suivi ces étapes le télescope ne se déplace toujours pas, il pourrait y avoir un problème matériel ou logiciel plus profond. Dans ce cas, il serait préférable de contacter le support technique de l'observatoire pour une assistance supplémentaire.

FIGURE 11 – Réponse du chatbot à la question du scénario 1

Scénario 2 – Fonctionnement du coronographe de Lyot

Objectif : Vérifier que le chatbot peut expliquer avec précision le fonctionnement du coronographe de Lyot et ses composants.

Question posée :

À quoi sert le cône obturateur dans le coronographe de Lyot ?

Réponse attendue :

Il explique que ce cône masque le disque solaire afin d'observer la couronne, et que plusieurs cônes sont nécessaires selon la distance Terre-Soleil.

Commentaire :

Le chatbot démontre sa capacité à restituer un contenu technique extrait d'un PDF avec justesse.

Réponse du chatbot :



Le cône obturateur dans le coronographe de Lyot sert à bloquer la lumière provenant du disque solaire. Il est positionné de manière à ce que sa base corresponde à l'image du disque solaire, ce qui permet d'éliminer l'essentiel du flux d'énergie incident. En obscurcissant une partie de l'image du disque solaire, le cône obturateur réduit ainsi l'énergie incidente transmise, permettant d'observer plus facilement la couronne solaire.

FIGURE 12 – Réponse du chatbot à la question du scénario 2

Scénario 3 – Différence entre télescopes Schmidt et Cassegrain

Objectif : Tester la capacité du bot à comparer deux systèmes optiques courants et identifier les différences physiques.

Question posée :

Quelle est la différence entre un miroir parabolique et un miroir sphérique dans un télescope ?

Réponse attendue :

Le bot indique que le miroir sphérique introduit une aberration corrigée par une lame de Schmidt, contrairement au miroir parabolique du Cassegrain.

Commentaire :

Ce scénario évalue la précision des connaissances scientifiques basées sur les documents fournis.

Réponse du chatbot :



La différence principale entre un miroir parabolique et un miroir sphérique dans un télescope réside dans leur forme et leur capacité à corriger les aberrations optiques.

Un miroir parabolique a une forme qui permet de concentrer les rayons lumineux parallèles en un point focal précis, ce qui en fait un choix idéal pour les télescopes car il minimise les aberrations sphériques. Cependant, un miroir parabolique est plus difficile et coûteux à fabriquer.

En revanche, un miroir sphérique est plus facile et moins cher à réaliser. Il a une forme sphérique qui peut également concentrer la lumière, mais il souffre d'aberrations de sphéricité, surtout lorsque l'on s'écarte des conditions de Gauss (conditions idéales de paraxialité). Ces aberrations peuvent être corrigées en utilisant une lame de Schmidt, comme c'est le cas dans un télescope Schmidt-Cassegrain.

En résumé, un miroir parabolique offre une meilleure qualité optique mais est plus coûteux, tandis qu'un miroir sphérique est plus économique mais nécessite des corrections pour obtenir des images de haute qualité.

FIGURE 13 – Réponse du chatbot à la question du scénario 3

Scénario 4 – Procédure de pointage dans Prism

Objectif : Tester l'intégration des procédures logicielles (ex. : Prism) dans le raisonnement du chatbot.

Question posée :

Comment pointer une étoile et synchroniser le télescope avec Prism ?

Réponse attendue :

Le chatbot explique la génération d'une carte du ciel, le double-clic sur l'objet, l'envoi de la commande de pointage et la synchronisation via le menu de Prism.

Commentaire :

Cela vérifie que le bot peut restituer fidèlement des séquences issues d'un manuel logiciel.

Réponse du chatbot :



Pour pointer une étoile et synchroniser le télescope avec Prism, suivez ces étapes :

1. Vérifier que l'onduleur, l'alimentation et le MCMT sont sous tension.
2. Dans l'onglet « observatoire » de la fenêtre principale de Prism, cliquer sur « établir lien matériel télescope ».
3. Choisir la position initiale du télescope : tube retourné ou à l'Est (côté parking) ou bien tube non retourné ou à l'Ouest (côté « jardin »). Une fois le choix effectué, cliquer sur « OK ».
4. Générer une carte du ciel dans Prism.
5. Pointer sur l'objet (l'étoile) en double-cliquant dessus dans la carte du ciel.
6. Cliquer sur « synchroniser avec l'objet » pour aligner le télescope avec l'étoile sélectionnée.

Ces étapes vous permettront de pointer une étoile et de synchroniser le télescope avec Prism.

FIGURE 14 – Réponse du chatbot à la question du scénario 4

Scénario 5 – Consignes de fin d'observation

Objectif : S'assurer que le chatbot peut rappeler correctement les consignes de fin de session à l'observatoire.

Question posée :

Que dois-je faire avant de fermer l'observatoire ?

Réponse attendue :

Il énumère la mise en parking, l'arrêt de la coupole, la coupure du lien matériel, l'extinction de la multiprise, le rangement et la fermeture.

Commentaire :

Scénario utile pour vérifier le respect des procédures opérationnelles courantes.

Réponse du chatbot :



Avant de fermer l'observatoire, vous devez suivre plusieurs étapes pour assurer la sécurité et le bon fonctionnement des équipements. Voici les principales actions à entreprendre :

1. **Dans le logiciel Prism :**
 - Allez dans l'onglet « observatoire » de la fenêtre principale de Prism.
 - Cliquez sur « couper lien matériel télescope » pour couper la communication entre le télescope et Prism.
2. **Éteindre l'électronique de pilotage :**
 - Éteindre le focuseur.
 - Éteindre le MCMT.
 - Éteindre l'alimentation.
 - Éteindre l'onduleur (dans cet ordre).
3. **Dans la tour d'observation :**
 - Fermer le volet de la coupole.
 - Ranger la console de commande et la raquette de pilotage.
 - Ranger les oculaires (ou autre montage optique) utilisés.
4. **Réorienter la coupole :**
 - Fermer le volet et réorienter la coupole volet à l'Ouest (côté « jardin »).
5. **Préparer le télescope :**
 - Remettre le cache du télescope.
 - Remettre le télescope au zénith est (côté « parking »).
6. **Dans la salle de commande :**
 - Éteindre le chauffage si il a été allumé.
 - Éteindre les lumières.
7. **Fermeture finale :**
 - Fermer la porte en partant.

En suivant ces étapes, vous vous assurez que l'observatoire est correctement fermé et que tous les équipements sont sécurisés.

FIGURE 15 – Réponse du chatbot à la question du scénario 5

Scénario 6 – Informations sur la caméra CCD

Objectif : S'assurer que le bot peut fournir des informations techniques précises sur l'équipement utilisé à l'observatoire

Question posée :

Quelle est la résolution de la caméra CCD utilisée à l'observatoire ?

Réponse attendue :

Le bot répond que la caméra SBIG STF-8300 dispose d'un capteur Kodak KAF-8300 avec une matrice de 3326×2504 pixels.

Commentaire :

Test simple de lecture d'information technique localisée dans un manuel PDF.

Réponse du chatbot :



La caméra CCD utilisée à l'observatoire a une résolution de 3326 x 2504 pixels, ce qui donne un total de 8.3 millions de pixels.

FIGURE 16 – Réponse du chatbot à la question du scénario 6

4.3. Pistes d'amélioration

- Support multimodal
- Interface enrichie

5. Conclusions et perspectives

5.1. Organisation et planification

- S1–2 : Cadrage
- S3–4 : Dev IA
- S5 : Interface + connexion
- S6–7 : Tests et intégration
- S8 : Revue finale

5.2. Bilan

Projet réussi : agent IA fonctionnel et utilisable.

5.3. Perspectives

Ajouts futurs : multimodalité, usage hors-ligne, déploiement élargi.

Bibliographie

- <https://docs.mistral.ai/guides/rag/>
- <https://huggingface.co/>
- <https://python.langchain.com/docs/>
- https://huggingface.co/learn/cookbook/en/multimodal_rag_using_document_retrieval_and_vlms

Annexes

Annexe 1 – Retour d'expérience individuel

Retour de Brahim ONASSER :

Ce projet PRONTO a représenté pour moi une expérience très enrichissante et unique. C'était la première fois que je travaillais sur un sujet aussi concret, avec un véritable enjeu pratique. Le fait de devoir collaborer régulièrement avec les encadrants ainsi qu'avec les responsables de l'observatoire m'a permis de me sentir plus impliqué et responsable. Cela m'a motivé à m'investir pleinement dans le projet et à m'organiser sérieusement à chaque étape.

Sur le plan technique, j'ai beaucoup appris, notamment sur les agents intelligents interactifs et leur conception. Ce projet m'a également permis de mieux comprendre comment créer des interfaces utilisateurs efficaces et intuitives, en tenant compte des besoins réels des utilisateurs.

Enfin, le travail en groupe a été un véritable point fort de cette expérience. Il m'a permis de développer mes compétences en collaboration, d'échanger des idées et de m'intégrer dans une dynamique collective. Chacun a apporté sa contribution, et j'ai appris à mieux écouter, proposer et coopérer au sein de l'équipe.

En résumé, ce projet m'a permis de progresser tant sur le plan technique que personnel, et je suis très satisfait de ce que nous avons accompli ensemble.

Retour d'Aymane OUNZAR :

Participer au projet PRONTO a été une expérience marquante qui m'a permis de sortir du cadre théorique pour me confronter à un vrai défi concret. C'était la première fois que je prenais part à une démarche aussi appliquée, où chaque étape avait un impact direct sur l'usage réel d'un outil.

Tout au long du projet, j'ai découvert des aspects nouveaux du travail en ingénierie logicielle, notamment dans la conception d'agents intelligents adaptés aux besoins d'utilisateurs spécifiques. J'ai aussi approfondi mes compétences en développement d'interfaces et en structuration de contenu interactif, en gardant toujours l'utilisateur au centre de mes réflexions.

Enfin, le travail collectif a été un vrai moteur dans cette aventure. J'ai appris à mieux collaborer, à prendre en compte différents points de vue et à m'intégrer dans une dynamique d'équipe où chacun avait un rôle important à jouer.

En résumé, cette expérience m'a permis de grandir à la fois sur le plan technique et humain, et elle restera pour moi un moment clé de mon parcours.

Retour de Mohamed Ali BOULLOUZ :

Participer au projet PRONTO a été pour moi une véritable immersion dans une démarche concrète et motivante. Concevoir un chatbot pour les visiteurs de l'observatoire de l'école nous a donné l'occasion de créer quelque chose de réellement utile, ce qui a complètement changé ma manière d'aborder un projet technique.

J'ai découvert l'importance d'analyser les besoins réels des utilisateurs avant de développer une solution. Cela m'a amené à réfléchir non seulement aux aspects techniques de l'agent conversationnel, mais aussi à la manière dont il serait perçu et utilisé sur le terrain. J'ai ainsi pu approfondir mes connaissances en conception d'interfaces simples et efficaces, mais aussi en structuration de dialogues pertinents.

Travailler en équipe a été un élément central de cette expérience. Nous avons dû apprendre à répartir les tâches, à communiquer clairement, à gérer nos différences de points de vue, et à nous ajuster en fonction des retours reçus. Cette collaboration m'a permis de mieux comprendre les dynamiques de groupe et l'importance d'un bon esprit d'équipe pour faire avancer un projet commun.

Avec du recul, je pense que ce projet m'a aidé à progresser autant sur le plan technique que dans ma manière de travailler avec les autres. J'en retiens une expérience très formatrice, qui m'a donné envie de continuer à m'impliquer dans des projets concrets ayant un impact direct.

Retour d'Axel JONARD :

Contexte : Dans le cadre du projet Pronto, notre objectif était de développer un agent intelligent basé sur des modèles de langage pour assister les utilisateurs d'un observatoire astronomique. Mon équipe était composée de membres ayant un niveau technique avancé, en particulier sur les aspects de programmation et d'implémentation de l'agent IA. De mon côté, je me suis naturellement tourné vers un rôle plus transversal, en m'assurant de la bonne progression globale du projet.

Action : J'ai pris l'initiative de veiller au bon respect des deadlines et à la cohérence de nos livrables. Pour cela, j'ai mis en place un planning partagé avec des jalons intermédiaires, et j'ai régulièrement relancé les membres du groupe pour suivre l'avancement. Je me suis également chargé de la rédaction des documents techniques et de la mise en forme finale des livrables (rapport, slides, etc.), en m'assurant qu'ils respectaient les consignes attendues. Cela a permis aux autres membres de se concentrer sur les aspects plus techniques du projet.

Résultat : Grâce à cette organisation, nous avons pu livrer un projet complet, bien structuré et dans les temps. Les rôles étaient clairs et chacun a pu s'appuyer sur ses forces. Cette prise de responsabilité m'a permis de développer des compétences en gestion de projet et en communication, tout en contribuant activement à la réussite collective.

3 CAMPUS, 1 SITE



Campus de Brest

Campus de Nantes

Campus de Rennes

2, rue de la Châtaigneraie
CS 17607
35576 Cesson Sévigné Cedex
France
T +33 (0)2 99 12 70 00
F +33 (0)2 51 85 81 99

Site de Toulouse

10, avenue Édouard Belin
BP 44004
31028 Toulouse Cedex 04
France
T +33 (0)5 61 33 83 65



Bretagne-Pays de la Loire
École Mines-Télécom

© IMT Atlantique, 2025
Imprimé à IMT Atlantique
Dépôt légal : Mai 2025
ISSN : 2556-5060